

第九章：硬件驱动

一、核心概念

1.1 设备驱动的角色

驱动是内核与硬件之间的翻译层。上层内核代码通过统一接口（read/write/ioctl）操作设备，驱动将这些操作转化为具体的硬件寄存器读写。

1.2 MMIO vs Port I/O

方式	架构	原理	访问方式
MMIO (Memory-Mapped I/O)	RISC-V, ARM, 现代 x86	设备寄存器映射到物理地址空间	普通 load/store
Port I/O	x86 传统	独立的 I/O 地址空间	in/out 指令

RISC-V 只有 MMIO。内核页表将设备物理地址恒等映射，通过 volatile 指针读写设备寄存器。

1.3 中断驱动 vs 轮询

方式	原理	优点	缺点
轮询 (Polling)	CPU 反复检查设备状态	简单，延迟低	浪费 CPU
中断驱动	设备完成后主动通知 CPU	CPU 可做其他事	中断开销、延迟不确定
DMA	设备直接读写内存	大块数据零 CPU 参与	需要物理地址，需要一致性管理

1.4 VirtIO 协议

VirtIO 是虚拟化环境中的标准设备接口，用于 Guest OS 与 Hypervisor 之间的高效通信。

核心数据结构：

- **Descriptor Table**：描述符数组，每项 = {物理地址, 长度, 标志, next}
- **Available Ring**：Guest 写，Host 读 ("这些 descriptor 链准备好了")
- **Used Ring**：Host 写，Guest 读 ("这些请求处理完了")

工作流：

```
Guest: 填 descriptor → 写 avail ring → MMIO notify (踢 Host)
Host:  读 avail ring → 处理请求 → 写 used ring → 触发中断
Guest: 中断处理 → 读 used ring → 完成回调
```

1.5 UART (16550)

最简单的字符设备，串口通信：

- **THR** (Transmit Holding Register): 写入要发送的字符
- **RHR** (Receive Holding Register): 读取收到的字符
- **LSR** (Line Status Register): 状态 (发送空闲/接收就绪)

1.6 中断控制器 PLIC

PLIC 负责将多个外设中断汇聚、排优先级、路由到指定 CPU 核:

- 每个中断源有优先级寄存器
- 每个 hart 有使能位 (选择接收哪些中断)
- **claim** 读取当前最高优先级待处理中断号
- **complete** 通知处理完毕

二、基本推论

推论 1: VirtIO 是理解真实驱动协议的绝佳入门。 VirtIO 的 descriptor chain + ring buffer 与真实 NVMe/网卡驱动的提交队列/完成队列本质相同。理解 VirtIO 可以迁移到任何 DMA 驱动。

推论 2: 中断处理必须快速。 PLIC claim 后到 complete 之间, 该中断源不会再次触发。长时间不 complete 会导致设备停止响应。

推论 3: MMIO 访问必须用 volatile。 编译器可能优化掉"看起来无用"的读写操作, 但设备寄存器的读写有副作用 (如清除中断标志)。volatile 防止编译器优化。

三、Linux / 通用实现

3.1 Linux 驱动模型

Linux 将驱动抽象为三层:

设备 (device)	↔	驱动 (driver)	↔	总线 (bus)
platform_device		platform_driver		platform_bus
pci_dev		pci_driver		pci_bus
usb_device		usb_driver		usb_bus

关键生命周期:

- **probe()**: 总线发现设备时调用, 驱动在此初始化硬件
- **remove()**: 设备移除时调用, 驱动在此清理资源

设备树 (Device Tree) : ARM/RISC-V 用设备树描述硬件拓扑, 内核解析后自动匹配 compatible 字符串找到对应驱动。

3.2 Linux VirtIO 子系统

```
struct virtio_driver {
    struct device_driver driver;
```

```
const struct virtio_device_id *id_table;
int (*probe)(struct virtio_device *dev);
void (*remove)(struct virtio_device *dev);
};

// virtqueue 操作
virtqueue_add_sgs(vq, sgs, out, in, data, GFP_ATOMIC);
virtqueue_kick(vq);
virtqueue_get_buf(vq, &len);
```

Linux 的 VirtIO 子系统封装了 ring buffer 操作，驱动只需提交 scatter-gather list。

3.3 Linux 中断申请

```
request_irq(irq, handler, flags, name, dev);
// flags: IRQF_SHARED (共享中断线)
//          IRQF_ONESHOT (线程化处理)

// 线程化中断 (现代推荐)
request_threaded_irq(irq, hardirq_handler, thread_fn, flags, name, dev);
```

四、RUOS 的实现

核心文件

- [src/devs/uart.c](#) — UART 串口
- [src/devs/console.c](#) — 控制台
- [src/devs/plic.c](#) — PLIC 中断控制器
- [src/virtIO/virtio_disk.c](#) — VirtIO 块设备
- [src/network/open-npstack/virtIO/virtio_net.c](#) — VirtIO 网卡

4.1 QEMU virt 平台设备地图

地址	设备	IRQ
0x10000000	UART0	10
0x10001000	VirtIO disk	1
0x10002000	VirtIO net	2
0x0C000000	PLIC	-
0x02000000	CLINT	- (M态定时器/IPI)

4.2 VirtIO 磁盘驱动

```
virtio_disk_rw(buf, write):
    1. 分配 3 个 descriptor:
        desc[0] = VirtIOBlkReq header (type, sector)
```

```

    desc[1] = 数据缓冲区
    desc[2] = 状态字节
2. 链接: desc[0]→desc[1]→desc[2]
3. 放入 avail ring
4. MMIO notify
5. sleep 等待中断
6. 中断来 → 检查 used ring → wakeup

```

对比 Linux: Linux 的 `virtio_blk` 驱动用 `virtqueue_add_sgs()` 抽象了 descriptor 操作, RUOS 手动管理 descriptor table——更底层但更清晰地展示了 VirtIO 协议的本质。

4.3 VirtIO 网卡

与磁盘的区别: 双 virtqueue (RX + TX)。RX 队列预填空缓冲区, Host 往里写收到的包。

4.4 PLIC 配置

```

plicinit():
    // 设置每个中断源的优先级
    PLIC[UART0_IRQ * 4] = 1;
    PLIC[VIRTIO0_IRQ * 4] = 1;

plicinithart():
    // 每个 hart 使能关心的中断源
    PLIC_SENABLE(hart) = (1<<UART0_IRQ) | (1<<VIRTIO0_IRQ) |
(1<<VIRTIO1_IRQ);

```

五、八股 / 面试高频问题

Q: DMA 是什么? 为什么需要物理地址? A: DMA (Direct Memory Access) 让设备直接读写内存, 无需 CPU 参与。设备不经过 MMU, 所以需要物理地址。Linux 通过 `dma_map_single()` 获取 DMA 地址, 处理 cache 一致性。

Q: 什么是 MMIO? 和 Port I/O 的区别? A: MMIO 将设备寄存器映射到物理地址空间, 用普通 load/store 访问。Port I/O 用独立 I/O 地址空间和 in/out 指令。MMIO 是现代标准, RISC-V/ARM 只有 MMIO。

Q: Linux 驱动的 probe 函数什么时候被调用? A: 当总线发现新设备且 `compatible/device_id` 与某个驱动匹配时, 内核自动调用该驱动的 `probe()`。probe 中初始化硬件、申请资源、注册设备节点。

Q: 什么是中断亲和性 (IRQ Affinity)? A: 将特定中断绑定到特定 CPU 核上处理, 减少 cache 抖动。通过 `/proc/irq/N/smp_affinity` 配置。

Q: VirtIO 的 split virtqueue 和 packed virtqueue 的区别? A: Split 用分离的 descriptor/avail/used 三个结构。Packed (VirtIO 1.1+) 将三者合并为单一环形缓冲区, 减少 cache miss, 提高性能。

六、项目贡献亮点

- 实现了 VirtIO 块设备驱动（descriptor chain + avail/used ring + 中断通知），对标 Linux virtio 子系统的 split virtqueue
- 实现了 VirtIO 网卡驱动（双队列 RX/TX），集成 TCP/IP 协议栈
- 理解 MMIO + PLIC 中断控制器的完整工作流

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
驱动模型	直接 MMIO 操作	设备树 + platform/pci driver	简单但不可扩展
VirtIO	手动管理 descriptor	virtqueue 抽象层	底层但教学价值高
中断处理	全在中断上下文	线程化 / 下半部	简单但延迟可能高
磁盘 I/O	同步等待（sleep）	异步 bio + 回调	简单但吞吐低